



Universidad Nacional
del Nordeste



Carrera: Licenciatura en Sistemas de Información

Asignatura: Redes de Datos

Alumno: Samuel Nehuen Zini

DNI: 44196064

LU: 56078

Tema N.º 3: TP 4.2 Programación de Sockets

INDICE

INDICE	2
DESARROLLO	4
Ejercicio 1:	4
1) Diseño y desarrollo aplicacion chat.	4
2) Aplicacion	4
3) Interfaz de usuario.	4
4) Retransmitir mensajes	5
5) Errores de conexion y desconexion	6
6) Implementa comandos especiales en el cliente.	6
7) Líneas de código.	7
8) código fuente del cliente y el servidor.	12
Ejercicio 2:	13
1) Nuevo Cliente de Chat	13
2) Inspección en Wireshark	14
FUENTES	17

Figura 1: prueba de chat python tcp 1.....	4
Figura 2: prueba de chat python tcp 2.....	5
Figura 3: prueba de chat python tcp 3.....	5
Figura 4: Excepciones lanzadas cuando ocurre un error al desconectar cliente.....	6
Figura 5: Pila de excepciones lanzadas.....	6
Figura 6: implementacion /quitar en cliente.....	6
Figura 7: implementacion /listar y /quitar en servidor.....	7
Figura 8: prueba usu1 chat tcp java.....	13
Figura 9: prueba usu2 chat tcp java.....	13
Figura 10: prueba servidor TCP java.....	14
Figura 11: wireshark observacion del chat con TCP.....	14
Figura 12: wireshark observacion del chat con UDP.....	14

DESARROLLO

Ejercicio 1:

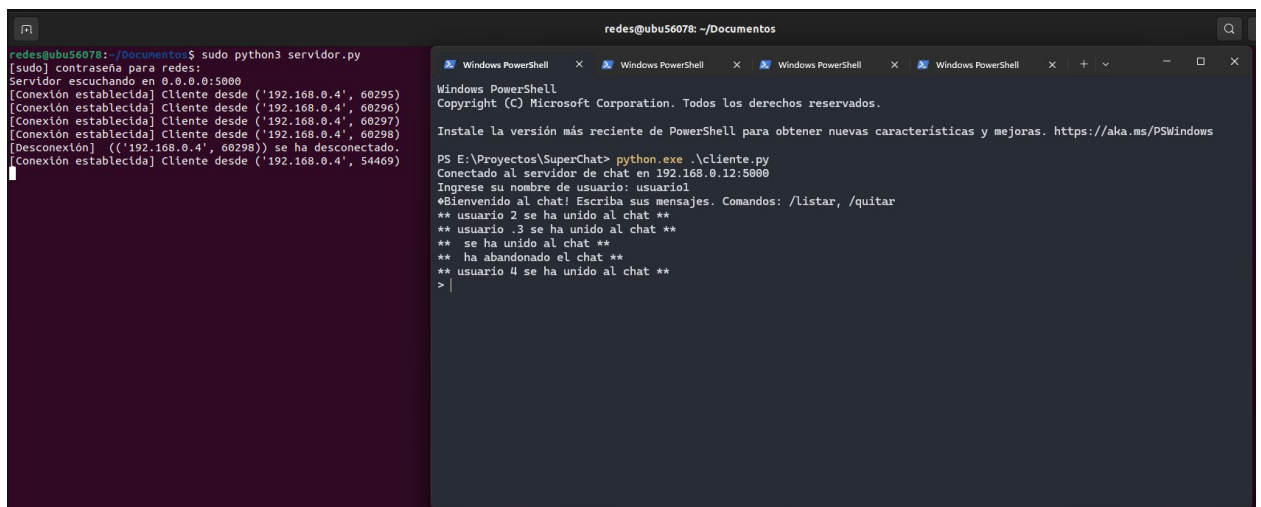
1) Diseño y desarrollo aplicacion chat.

Para diseñar un servicio de mensajería por sockets, es importante entender que los sockets conectan la capa de red con la de aplicación y existen dos tipos principales según el protocolo: TCP, que es confiable y orientado a conexión, garantiza la entrega y el orden de los datos mediante un proceso de handshake; y UDP, que es más rápido y sencillo pero no garantiza la entrega ni el orden, enviando mensajes sin confirmación. Para la comunicación, se necesitan direcciones IP y números de puerto que identifican las máquinas y aplicaciones, respectivamente. En TCP, cada conexión se identifica por una combinación única de direcciones IP y puertos de servidor y cliente.

2) Aplicacion

A continuacion se puede ver verificaicones de la aplicacion funcionando.

3) Interfaz de usuario.



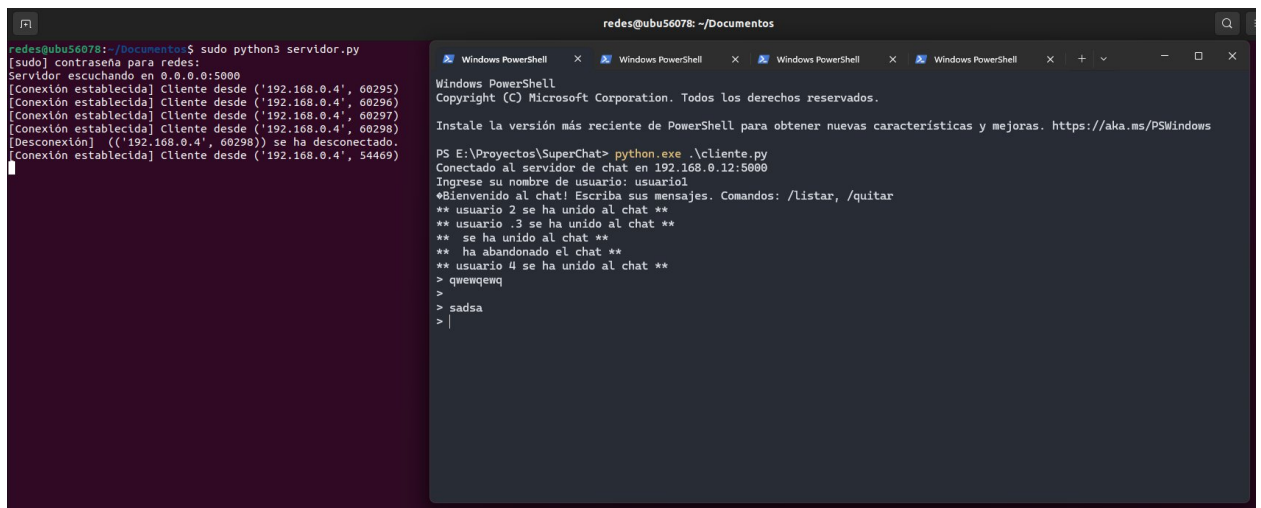
```
redes@ubu56078: ~/Documentos
redes@ubu56078:~/Documentos$ sudo python3 servidor.py
[sudo] contraseña para redes:
Servidor escuchando en 0.0.0.0:5000
[Conexión establecida] Cliente desde ('192.168.0.4', 60295)
[Conexión establecida] Cliente desde ('192.168.0.4', 60296)
[Conexión establecida] Cliente desde ('192.168.0.4', 60297)
[Conexión establecida] Cliente desde ('192.168.0.4', 60298)
[Desconexión] (('192.168.0.4', 60298)) se ha desconectado.
[Conexión establecida] Cliente desde ('192.168.0.4', 54469)

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS E:\Proyectos\SuperChat> python.exe .\cliente.py
Conectado al servidor de chat en 192.168.0.12:5000
Ingrese su nombre de usuario: usuariol
*Bienvenido al chat! Escriba sus mensajes. Comandos: /listar, /quitar
** usuario 2 se ha unido al chat **
** usuario .3 se ha unido al chat **
** se ha unido al chat **
** ha abandonado el chat **
** usuario 4 se ha unido al chat **
> |
```

Figura 1: prueba de chat python tcp 1



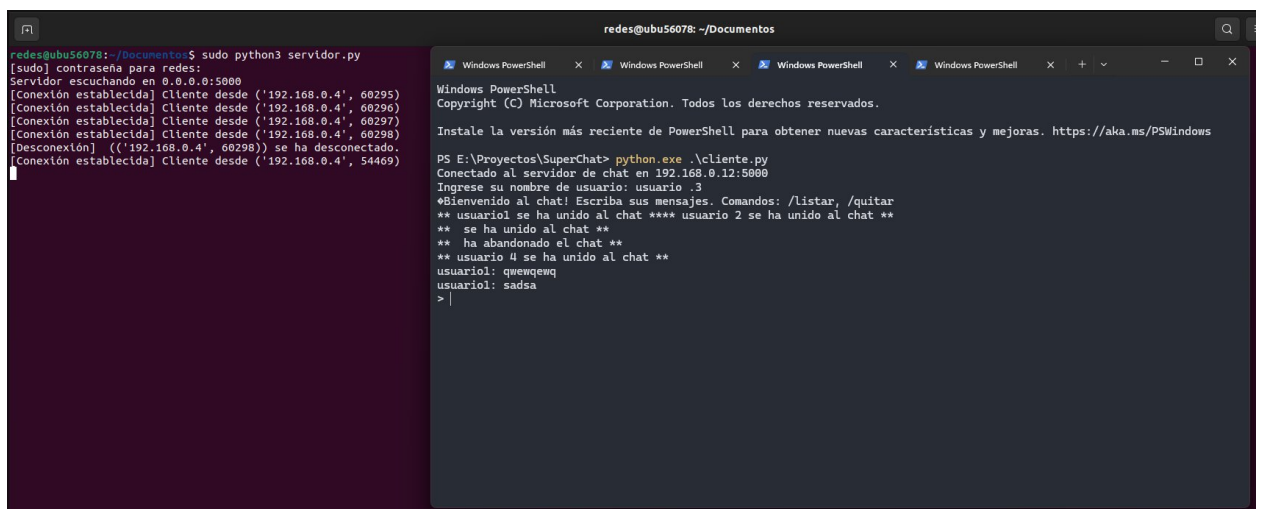
```
redes@ubu56078: ~/Documentos
[sudo] contraseña para redes:
Servidor escuchando en 0.0.0.0:5000
[Conexión establecida] Cliente desde ('192.168.0.4', 60295)
[Conexión establecida] Cliente desde ('192.168.0.4', 60296)
[Conexión establecida] Cliente desde ('192.168.0.4', 60297)
[Conexión establecida] Cliente desde ('192.168.0.4', 60298)
[Desconexión] (('192.168.0.4', 60298)) se ha desconectado.
[Conexión establecida] Cliente desde ('192.168.0.4', 54469)

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS E:\Proyectos\SuperChat> python.exe .\cliente.py
Conectado al servidor de chat en 192.168.0.12:5000
Ingrese su nombre de usuario: usuario1
*Bienvenido al chat! Escriba sus mensajes. Comandos: /listar, /quitar
** usuario 2 se ha unido al chat **
** usuario .3 se ha unido al chat **
** se ha unido al chat **
** ha abandonado el chat **
** usuario 4 se ha unido al chat **
> qwewqewq
>
> sadsa
>
> |
```

Figura 2: prueba de chat python tcp 2



```
redes@ubu56078: ~/Documentos
[sudo] contraseña para redes:
Servidor escuchando en 0.0.0.0:5000
[Conexión establecida] Cliente desde ('192.168.0.4', 60295)
[Conexión establecida] Cliente desde ('192.168.0.4', 60296)
[Conexión establecida] Cliente desde ('192.168.0.4', 60297)
[Conexión establecida] Cliente desde ('192.168.0.4', 60298)
[Desconexión] (('192.168.0.4', 60298)) se ha desconectado.
[Conexión establecida] Cliente desde ('192.168.0.4', 54469)

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS E:\Proyectos\SuperChat> python.exe .\cliente.py
Conectado al servidor de chat en 192.168.0.12:5000
Ingrese su nombre de usuario: usuario .3
*Bienvenido al chat! Escriba sus mensajes. Comandos: /listar, /quitar
** usuario1 se ha unido al chat *** usuario 2 se ha unido al chat **
** se ha unido al chat **
** ha abandonado el chat **
** usuario 4 se ha unido al chat **
usuario1: qwewqewq
usuario1: sadsa
>
> |
```

Figura 3: prueba de chat python tcp 3

4) Retransmitir mensajes

Los mensajes de los hosts son recibidos por el servidor y son retransmitidos por broadcasts a los demás hosts, a cada host le es asignado un hilo propio de procesador.

5) Errores de conexion y desconexion

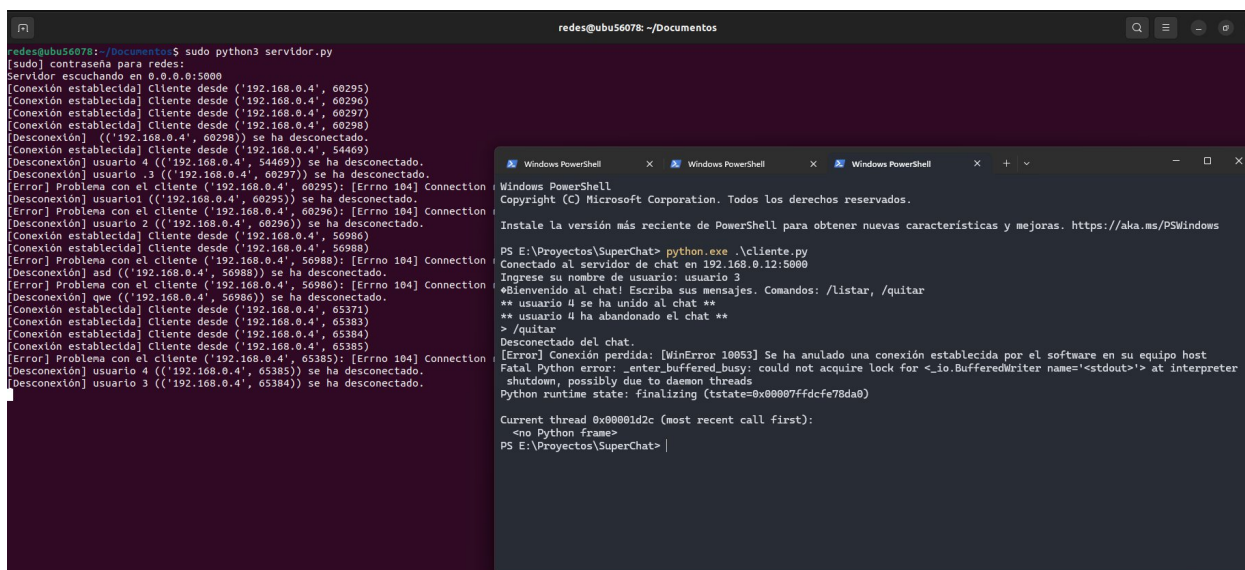


Figura 4: Excepciones lanzadas cuando ocurre un error al desconectar cliente.

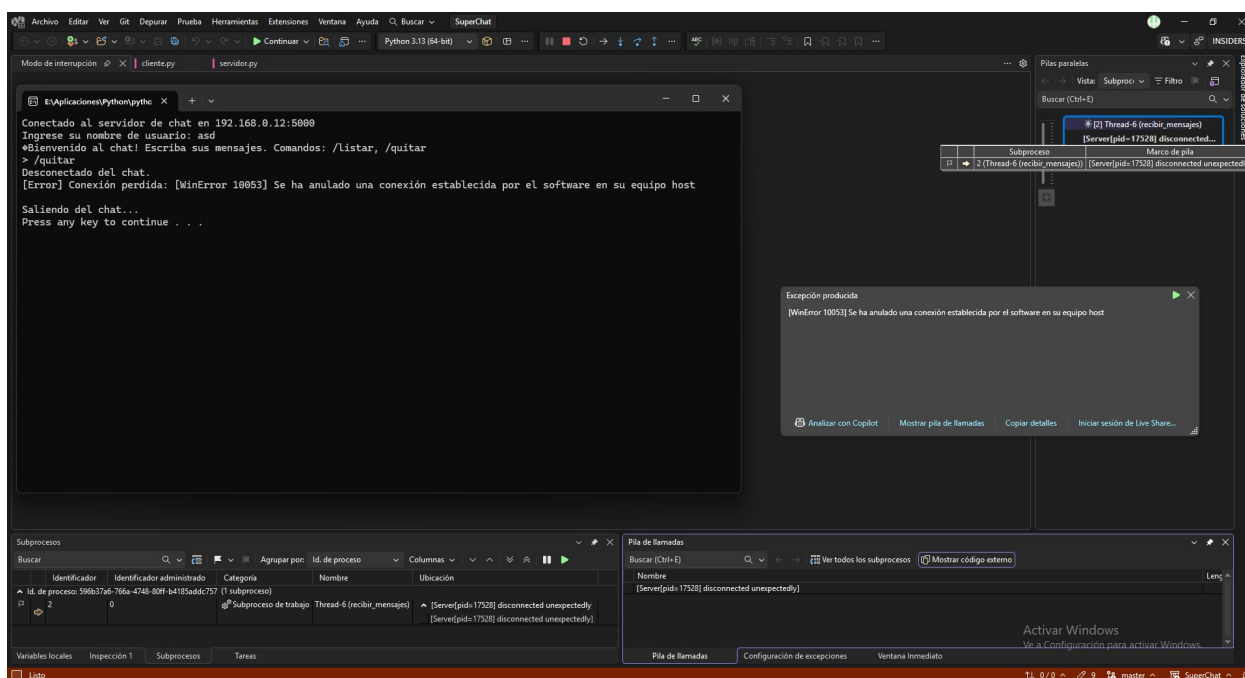


Figura 5: Pila de excepciones lanzadas.

La excepcion parece estar en la parte de hilos, puede que al cerrar la comunicacion no se cierran correctamente los hilos de comunicacion. Tambien se puede ver la excepcion que salta en el servidor cuando un cliente se desconecta de forma incorrecta.

6) Implementa comandos especiales en el cliente.

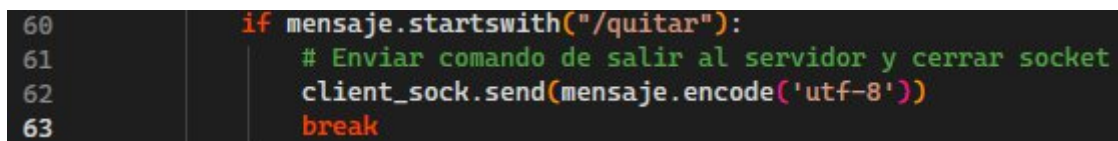


Figura 6: implementacion /quitar en cliente

```

40         if mensaje.startswith('/listar'):
41             # Preparar lista de usuarios conectados
42             lista_usuarios = ", ".join(clients_names.values())
43             conn.send(f"Usuarios conectados: {lista_usuarios}\n".encode('utf-8'))
44         elif mensaje.startswith('/quitar'):
45             # El cliente pidió desconectarse
46             break # salimos del bucle para cerrar conexión

```

Figura 7: implementación /listar y /quitar en servidor.

7) Líneas de código.

Primero, creamos un socket TCP y lo vinculamos a una dirección y puerto donde escuche las conexiones:

```

import socket
import threading

HOST = '0.0.0.0' # Escuchar en todas las interfaces de red (0.0.0.0)
PORT = 5000      # Puerto de escucha para el chat (elegido arbitrariamente)

server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM) #Se crea un socket
TCP con socket.AF_INET (familia IPv4) y socket.SOCK_STREAM (tipo TCP)
server_sock.bind((HOST, PORT))
server_sock.listen(5) # Cola de hasta 5 conexiones pendientes
print(f"Servidor escuchando en {HOST}:{PORT}")

```

Se crea un socket TCP con socket.AF_INET (familia IPv4) y socket.SOCK_STREAM (tipo TCP). Luego se bindea al host/puerto deseados. Aquí usamos HOST = '0.0.0.0' para aceptar conexiones desde cualquier interfaz de red disponible (si solo se quisiera localmente, se usaría 'localhost' o '127.0.0.1'). El puerto 5000 es un puerto arbitrario no privilegiado (>1023) para nuestro servicio de chat; en un entorno real, asegurarse de que esté libre o documentarlo. Finalmente, listen(5) pone al socket en modo escucha, permitiendo hasta 5 conexiones en espera (este número puede ajustarse). Ahora el servidor está listo para aceptar clientes entrantes.

El servidor entra en un bucle infinito aceptando nuevas conexiones con accept(). Cada vez que un cliente se conecta, accept() devuelve un nuevo socket y la dirección del cliente. En ese momento creamos un hilo para gestionar a ese cliente y continuamos esperando otros.

```

import threading

clients = [] # lista de sockets de clientes conectados
clients_names = {} # diccionario para mapear socket de cliente a su nombre de usuario

def atender_cliente(conn, addr):
    # Función que se ejecutará en un hilo para atender a un cliente en particular.
    print(f"[Conexión establecida] Cliente desde {addr}")
    try:
        # Primero, recibir el nombre de usuario que el cliente envía como identificación
        username = conn.recv(1024).decode('utf-8')
        if not username:
            conn.close()
            return
        username = username.strip()
        clients_names[conn] = username
        # Notificar a todos los demás que este usuario se ha unido (opcional)
        mensaje_bienvenida = f"*** {username} se ha unido al chat ***"
        _broadcast(mensaje_bienvenida, conn)

```

```

# Bucle principal de recepción de mensajes de este cliente
while True:
    data = conn.recv(1024)
    if not data:
        # Si data viene vacío, el cliente cerró la conexión
        break
    mensaje = data.decode('utf-8').strip()
    if mensaje == "":
        continue # ignorar mensajes vacíos (por ejemplo solo ENTER)
    # Manejo de comandos especiales:
    if mensaje.startswith('/listar'):
        # Preparar lista de usuarios conectados
        lista_usuarios = ", ".join(clients_names.values())
        conn.send(f"Usuarios conectados: {lista_usuarios}\n".encode('utf-8'))
    elif mensaje.startswith('/quitar'):
        # El cliente pidió desconectarse
        break # salimos del bucle para cerrar conexión
    else:
        # Mensaje normal: reenviarlo a todos los demás clientes
        mensaje_formateado = f"{username}: {mensaje}"
        _broadcast(mensaje_formateado, conn)
except Exception as e:
    print(f"[Error] Problema con el cliente {addr}: {e}")
finally:
    # Cleanup al desconectarse el cliente
    if conn in clients:
        clients.remove(conn)
        nombre = clients_names.get(conn, "<desconocido>")
        print(f"[Desconexión] {nombre} ({addr}) se ha desconectado.")
        # Notificar a los demás que el usuario salió (opcional)
        _broadcast(f"*** {nombre} ha abandonado el chat ***", conn)
        clients_names.pop(conn, None)
        conn.close()

def _broadcast(mensaje, origen_conn=None):
    """Envía el mensaje a todos los clientes conectados, excepto al de origen (si se indica)."""
    data = mensaje.encode('utf-8')
    for client in clients:
        if client is origen_conn:
            continue
        try:
            client.send(data)
        except Exception as e:
            print(f"[Advertencia] No se pudo enviar a un cliente: {e}")

```

```

while True:
    conn, addr = server_sock.accept() # Esperar una nueva conexión
    clients.append(conn)
    # Crear y lanzar un hilo para atender al nuevo cliente
    hilo = threading.Thread(target=atender_cliente, args=(conn, addr), daemon=True)
    hilo.start()

```

Aquí definimos la función `atender_cliente(conn, addr)` que será ejecutada en un nuevo hilo por cada conexión aceptada. Este hilo se encarga de comunicarse con un cliente específico.

Al entrar, lee el nombre de usuario enviado por el cliente como primera acción. Suponemos que el cliente, tras conectarse, envía su nombre (este detalle de protocolo lo definiremos para identificar usuarios). Al decodificar username, lo almacenamos en `clients_names` dict vinculado al socket del cliente. También podemos informar al resto del chat que el usuario se unió, enviando un mensaje de sistema (por ejemplo `** usuario se ha unido **`).

Luego entra en un bucle `while True` para recibir continuamente mensajes del cliente via `conn.recv(1024)`. Usamos un tamaño de buffer de 1024 bytes (suficiente para mensajes cortos; si se quisieran mensajes más largos, se podría usar más).

Si `recv` devuelve `not data` (es decir, una cadena vacía `b''`), significa que el cliente cerró la conexión y salimos del bucle para terminar el hilo.

Convertimos los datos a cadena con `decode('utf-8')` (los sockets transmiten bytes, aquí asumimos texto UTF-8). Realizamos `strip()` para quitar saltos de línea o espacios sobrantes.

Si el mensaje no es vacío, verificamos si inicia con nuestros comandos especiales:

`/listar`: el servidor compila la lista de nombres de usuarios conectados (iterando `clients_names.values()`) y envía esa lista de vuelta solo al cliente que lo pidió, usando `conn.send()`. Nótese que codificamos la cadena a bytes. Este mensaje aparecerá en la consola del cliente como respuesta.

`/quitar`: simplemente hacemos `break` del bucle, lo que conducirá a cerrar la conexión con ese cliente (interpretamos que el propio cliente también se cerrará voluntariamente tras enviar este comando).

Cualquier otro texto se trata como mensaje de chat normal. Entonces formamos una cadena con el formato `"username: mensaje"` para identificar al remitente, y llamamos a `_broadcast(mensaje_formateado, conn)` para enviarlo a todos los demás clientes.

La función `_broadcast` recorre la lista `clients` (sockets conectados) y envía el mensaje a cada uno excepto, opcionalmente, al que corresponde al `origen_conn` (de donde vino el mensaje). Esto coincide con la lógica esperada: reenviar a todos los demás clientes conectados el mensaje recibido. Si ocurre un error al enviar a algún cliente (por ejemplo, si un cliente se desconectó repentinamente), capturamos la excepción y opcionalmente removemos ese cliente de la lista.

Usamos bloque `try/except` alrededor de la recepción y envío para capturar cualquier error de I/O. Si hay una excepción inesperada, se imprime un mensaje de error pero el hilo procedería a finalizar limpiamente en el `finally`.

En el bloque `finally`, se realiza la limpieza al terminar la comunicación con un cliente: eliminar su socket de la lista `clients` y su nombre del diccionario `clients_names`. También informamos por consola del servidor la desconexión. Podemos enviar un mensaje a los demás anunciando que ese usuario salió (pasando `origen_conn` como el socket que se desconectó, para que `_broadcast` omita enviárselo a sí mismo, aunque ese socket ya no está en `clients` de todos modos). Finalmente se cierra el socket con `conn.close()`.

La creación del hilo se hace con `threading.Thread(target=atender_cliente, args=(conn, addr), daemon=True)` y luego `start()` para que comience a correr concurrentemente. Marcamos el hilo como `daemon` (opcional) para que no impida salir del programa si todos los hilos principales terminan. El servidor principal luego vuelve al inicio del bucle `while True` y llama a `accept()` nuevamente quedando a la espera de otra conexión entrante sin bloquear el manejo de los ya

conectados. Con este diseño, el servidor puede gestionar N clientes simultáneamente (limitado por los recursos del sistema), delegando la I/O de cada uno a hilos separados.

Primero, el cliente debe establecer la conexión TCP con el servidor:

```
import socket
import threading

SERVER_HOST = '127.0.0.1' # IP o nombre de host del servidor (localhost para pruebas)
SERVER_PORT = 5000        # Puerto en el que escucha el servidor de chat

# Crear socket TCP del cliente
client_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
try:
    client_sock.connect((SERVER_HOST, SERVER_PORT))
    print(f"Conectado al servidor de chat en {SERVER_HOST}:{SERVER_PORT}")
except Exception as e:
    print(f"No se pudo conectar al servidor: {e}")
    exit(1)
```

Aquí se configura el socket de manera similar, pero usando connect() en lugar de bind/listen. Si el servidor está en otra máquina, SERVER_HOST debe ser la IP de esa máquina (por ejemplo la IP de la VM Ubuntu en la red local). Tras connect, la conexión TCP de cliente a servidor queda establecida (realiza internamente el handshake TCP).

El cliente envía inmediatamente su nombre de usuario al servidor, para que este lo registre:

```
nombre_usuario = input("Ingrese su nombre de usuario: ")
if not nombre_usuario:
    nombre_usuario = "Anonimo"
client_sock.send(nombre_usuario.encode('utf-8'))
```

El servidor espera esto como primer mensaje para asociar el socket a un nombre.

Para poder mostrar mensajes entrantes del chat en cualquier momento, creamos un hilo que se encargue únicamente de recibir datos del servidor e imprimirlos en pantalla, mientras el hilo principal se dedicará a leer la entrada del usuario

```
# Función que ejecutará el hilo de recepción
def recibir_mensajes():
    try:
        while True:
            data = client_sock.recv(1024)
            if not data:
                # El servidor cerró la conexión
                print("*** Conexión cerrada por el servidor ***")
                break
            mensaje = data.decode('utf-8').strip()
            if mensaje:
                print("\r" + mensaje + "\n> ", end="")
                # Imprime el mensaje recibido. "\r" retorna al inicio de línea.
```

```

        # Luego muestra el prompt ">" nuevamente para el usuario.
except Exception as e:
    print(f"[Error] Conexión perdida: {e}")
finally:
    # Salir del programa cuando se pierde la conexión
    client_sock.close()
    print("\nSaliendo del chat...")
    os._exit(0) # fuerza la salida de todo el proceso

# Iniciar el hilo receptor
hilo_receptor = threading.Thread(target=recibir_mensajes, daemon=True)
hilo_receptor.start()

```

La función `recibir_mensajes` corre en segundo plano. Usa un bucle `recv` para esperar datos del servidor. Cada vez que llega algo, lo decodifica y lo imprime. Notemos la sintaxis de impresión: usamos `"r"` al inicio y luego `"\n> "` al final para redibujar un prompt `>` de entrada. Esto es estético, para que si el usuario estaba escribiendo, el mensaje entrante aparezca y luego se muestre de nuevo el prompt listo para seguir escribiendo en la nueva línea.

Si `recv` devuelve vacío, significa que el servidor cortó la conexión (por ejemplo, se cayó el servidor o cerró). Entonces se informa y se sale del bucle. En el `finally`, cerramos el socket y terminamos el programa (`os._exit(0)` finaliza inmediatamente el proceso, asegurando que incluso si el hilo principal estaba esperando input, se detenga). Este mecanismo garantiza que el cliente no quede colgado si el servidor cae.

Marcamos el hilo receptor como `daemon` para que no impida salir del programa; aunque usamos `_exit` para forzar salida ante desconexión, el hilo `daemon` aseguraría terminar cuando el `main thread` acaba.

Con el hilo receptor funcionando, el hilo principal del cliente puede ahora manejar la entrada del usuario sin preocuparse de los mensajes entrantes. Implementamos un loop para leer líneas de la entrada estándar y enviarlas al servidor.

```

print("¡Bienvenido al chat! Escriba sus mensajes. Comandos: /listar, /quitar")
while True:
    try:
        mensaje = input("> ")
    except EOFError:
        # Si se presiona Ctrl+D (EOF) terminar
        mensaje = "/quitar"
    if mensaje.strip() == "":
        continue # ignorar líneas vacías
    if mensaje.startswith("/quitar"):
        # Enviar comando de salir al servidor y cerrar socket
        client_sock.send(mensaje.encode('utf-8'))
        break
    try:
        client_sock.send(mensaje.encode('utf-8'))
    except Exception as e:
        print(f"[Error] enviando mensaje: {e}")
        break

# Si salimos del bucle, cerramos el socket y terminamos

```

```
client_sock.close()
print("Desconectado del chat.")
```

Mostramos un mensaje de bienvenida con indicaciones de los comandos. Luego en un bucle infinito usamos `input("> ")` para leer la línea que escriba el usuario (mostrando un prompt `>`). Si el usuario solo aprieta Enter sin teclear nada (cadena vacía), simplemente continuamos el loop (no enviamos mensajes vacíos).

Si lee EOF (por ejemplo, usuario presiona Ctrl+D en Linux), interpretamos eso como deseo de salir y forzamos mensaje = `"/quitar"`.

Cuando el mensaje comienza con `/quitar`, enviamos ese comando al servidor (para notificar la salida) y hacemos break para salir del loop local. Así no pediremos más input y pasaremos a cerrar todo.

Para cualquier otro mensaje (incluyendo `/listar` u otro texto normal), simplemente enviamos el texto al servidor usando `client_sock.send()`. Está dentro de un try/except por si falla la conexión en ese instante.

Si ocurre una excepción al enviar, se imprime un error y se sale del bucle, lo que conducirá también a cerrar el socket.

Tras salir del bucle (que ocurre solo en caso de desconexión voluntaria o error), cerramos el socket con `close()` y avisamos al usuario.

Gracias al hilo receptor, los mensajes de respuesta del servidor (como la lista de usuarios o los mensajes de otros clientes) se seguirán imprimiendo incluso mientras uno está en `input()`. En Python, la función `input()` bloquea la entrada, pero la impresión de mensajes de fondo puede hacerse visible. Nuestra forma de imprimir con `"\r"` intenta que quede prolijo (aunque en consola pura puede haber ligeras incongruencias de prompt, es un detalle menor).

8) código fuente del cliente y el servidor.

Se puede encontrar la aplicación hecha con python en el siguiente repositorio github: <https://github.com/CodinRes/SuperChatRedes>. La aplicación funciona sobre un CLI, el archivo `servidor.py` se coloca dentro de la máquina servidor y el archivo `cliente.py` es utilizado por los múltiples hosts, se ejecutan con python.

Ejercicio 2:

1) Nuevo Cliente de Chat

Se elige java, a continuación se puede ver verificaciones del funcionamiento.

```
Ingrese la IP del servidor: 123
No se pudo conectar al servidor: Network is unreachable: connect
PS E:\Proyectos\SuperChat> ^C
PS E:\Proyectos\SuperChat>
PS E:\Proyectos\SuperChat> e:: cd 'e:\Proyectos\SuperChat'; & 'E:\
paceStorage\A28ebce579b853dda5d866af5777cac2\redhat.java\jdt_ws\
Ingrese la IP del servidor: 192.168.0.12
Conectado al servidor de chat en 192.168.0.12:5000
Ingrese su nombre de usuario: wqewq
¡Bienvenido al chat! Comandos: /listar, /quitar
> /listar
>
Usuarios conectados: wqewq
>
** asdas se ha unido al chat **
>
** zxcxz se ha unido al chat **
>
zxcxz: asdsafdgfgf
>
asdas: gertre
>
** asdas ha abandonado el chat **
> []
```

Figura 8: prueba usu1 chat tcp java

```
Ingrese la IP del servidor: 192.168.0.12
Conectado al servidor de chat en 192.168.0.12:5000
Ingrese su nombre de usuario: asdas
¡Bienvenido al chat! Comandos: /listar, /quitar
>
** zxcxz se ha unido al chat **
>
zxcxz: asdsafdgfgf
> gertre
> /listar
>
Usuarios conectados: wqewq, asdas, zxcxz
> /quitar
Desconectado del chat.
PS E:\Proyectos\SuperChat> []
```

Figura 9: prueba usu2 chat tcp java

```
redes@ubu56078:~/Documentos$ java ServidorTCP
Servidor TCP escuchando en el puerto 5000
[Conectado] wqwqw desde /192.168.0.4:61851
[Conectado] asdas desde /192.168.0.4:61862
[Conectado] zxcxz desde /192.168.0.4:55150
[Desconectado] asdas (/192.168.0.4:61862)
```

Figura 10: prueba servidor TCP java

2) Inspección en Wireshark

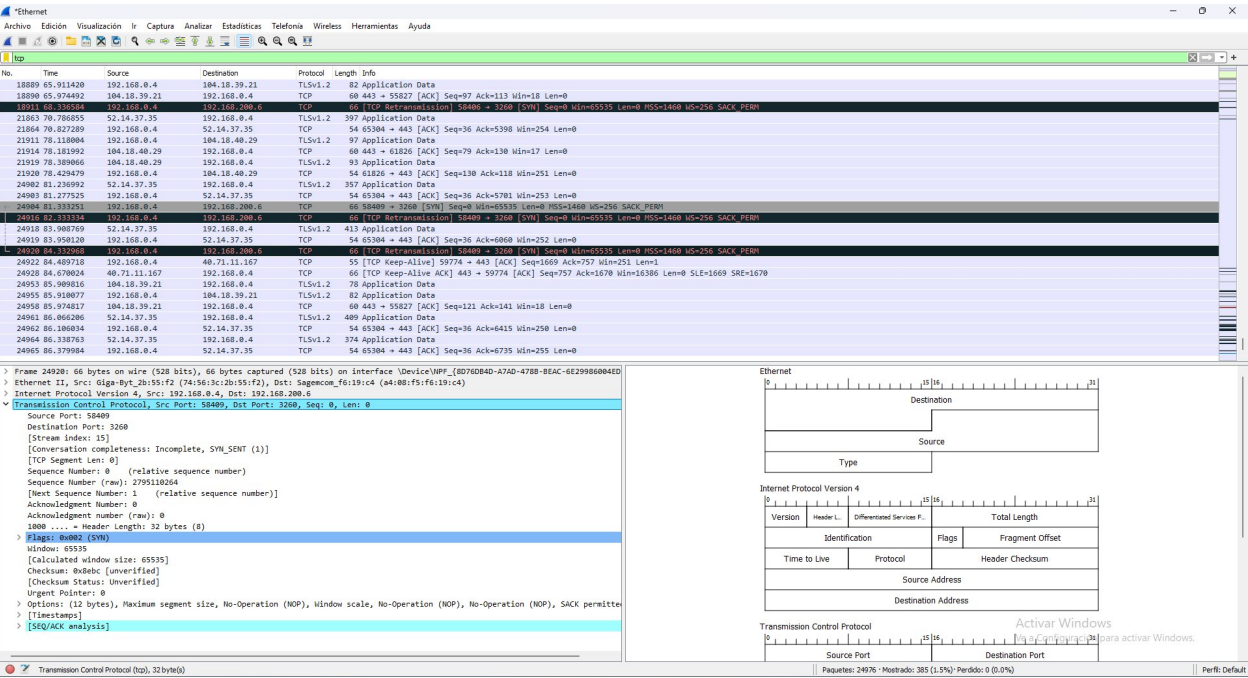


Figura 11: wireshark observacion del chat con TCP

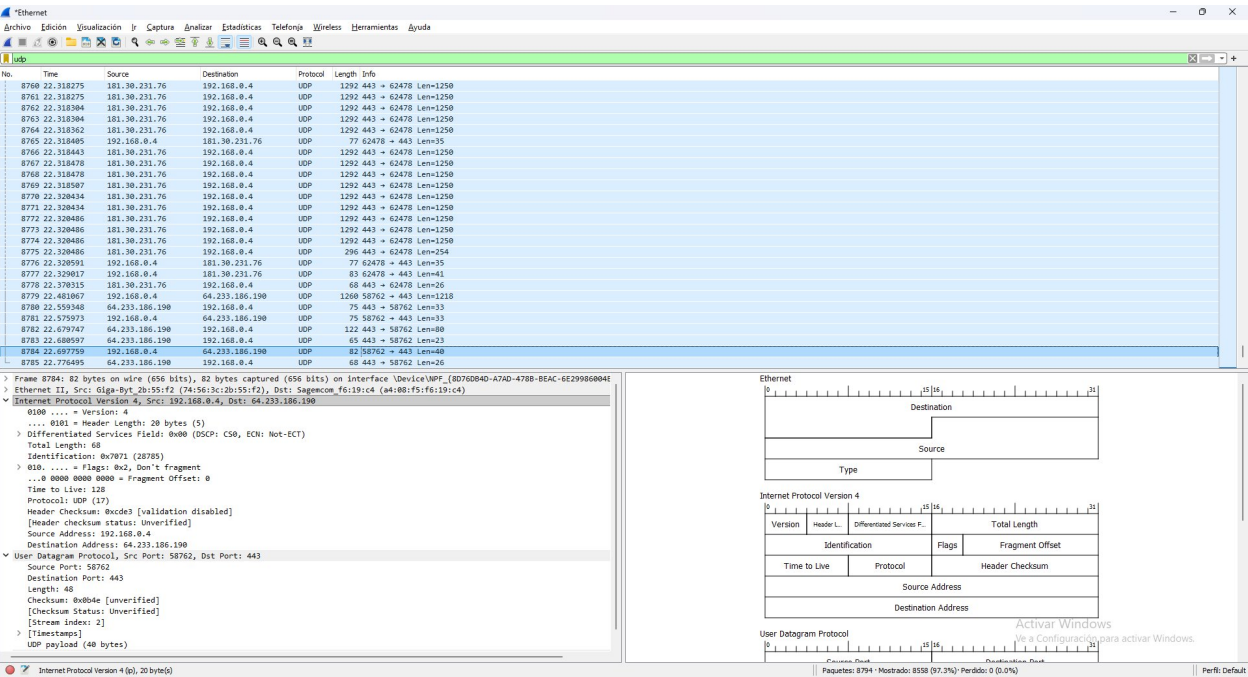


Figura 12: wireshark observacion del chat con UDP

En la primera captura se observa claramente cómo opera TCP (Transmission Control Protocol). Wireshark muestra los siguientes elementos clave:

Establecimiento de conexión (Three-Way Handshake):

Se observan tres paquetes consecutivos con los flags: [SYN] → [SYN, ACK] → [ACK]

Este intercambio inicial confirma que TCP es un protocolo orientado a conexión, que establece una sesión antes de enviar datos. Ejemplo del paquete analizado:

Transmission Control Protocol

Source Port: 58409

Destination Port: 3260

Flags: 0x002 (SYN)

Aquí el cliente inicia la conexión enviando un paquete SYN al puerto 3260 del servidor. El servidor responde con SYN, ACK, y finalmente el cliente confirma con un ACK, completando la conexión.

Campos del encabezado TCP:

En la parte inferior de Wireshark se muestran los campos típicos:

Source Port y Destination Port: indican los puertos de origen y destino.

Sequence Number y Acknowledgment Number: numeran los bytes para mantener el orden y control de la transmisión.

Flags: SYN, ACK, PSH, FIN, etc., que controlan el estado de la conexión.

Window size: tamaño de la ventana de recepción, relacionado con el control de flujo.

Retransmisión y control de errores:

En la captura aparecen líneas marcadas como "TCP Retransmission", lo que evidencia que TCP detecta pérdidas de paquetes y los reenvía automáticamente, garantizando fiabilidad en la comunicación.

Cierre de conexión:

Al finalizar la sesión, se observan paquetes con las banderas FIN y ACK, que corresponden al cuatro pasos de cierre de conexión (Four-way handshake).

Análisis del tráfico UDP

En la segunda captura se analizó una comunicación con el protocolo UDP (User Datagram Protocol).

Se observa una lista de paquetes UDP consecutivos, sin handshakes ni confirmaciones.

Transmisión sin conexión:

A diferencia de TCP, UDP no realiza ningún intercambio previo de conexión. El cliente simplemente envía datagramas directamente al servidor.

Ejemplo de paquete:

User Datagram Protocol
Source Port: 58762
Destination Port: 443
Length: 68

Cada datagrama incluye la dirección IP y puerto de destino, y Wireshark los muestra como tramas independientes.

Estructura del encabezado UDP:

En el análisis del paquete se identifican los campos:

Source Port
Destination Port
Length: tamaño total del datagrama UDP.
Checksum: verificación básica de errores (opcional).

No existen números de secuencia ni confirmaciones (ACK), lo que indica que UDP no garantiza la entrega ni el orden de los mensajes.

Ausencia de control de flujo o retransmisión:

No se observan retransmisiones ni mensajes de confirmación.

Esto evidencia la baja sobrecarga y la velocidad superior de UDP, aunque con la desventaja de que los mensajes podrían perderse o llegar desordenados.

FUENTES

<https://www.datacamp.com/es/tutorial/a-complete-guide-to-socket-programming-in-python>

https://miot-rpi.github.io/practicas/RPI-II/P1_IV/#:~:text=el servidor echo TCP y,aspecto que consideres de inter%C3%A9s

[https://www.redeszone.net/tutoriales/configuracion-puertos/puertos-tcp-udp/#:~:text=Tanto en TCP como en UDP tenemos,volver\) hace uso de un puerto ef%C3%ADmero.](https://www.redeszone.net/tutoriales/configuracion-puertos/puertos-tcp-udp/#:~:text=Tanto en TCP como en UDP tenemos,volver) hace uso de un puerto ef%C3%ADmero.)